Lab1: OpenCV Intro

Computer Vision Course — A.A. 2025/2026

Giulia Martinelli

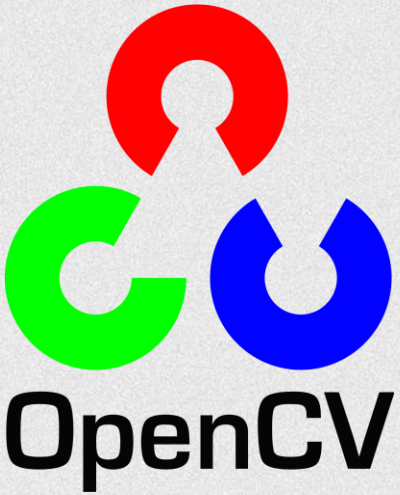
giulia.martinelli-2@unitn.it

What is OpenCV?

OpenCV (Open Source Computer Vision Library: http://opencv.org) is an open-source library that includes several hundreds of computer vision algorithms

- Image processing
- Video Analysis
- Camera Calibration and 3D Reconstruction
- Object Detection
- ... and more!

- Website: opencv.org
- Docs: docs.opencv.org
- Tutorials



What are we gonna see today??

- How to open and display images
- How to open and display videos
- Calculate an histogram
- Edge Detection
- Filtering
- Morphological transformations

How to open and display images

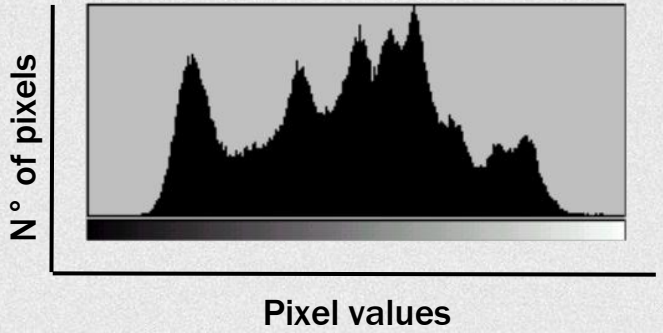
How to open and display videos

Histograms

- **BINS:** the number of pixels for every pixel value
- **DIMS:** number of parameters for which we collect the data
- **RANGE:** range of intensity values

cv.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]])

- **images:** image path
- **channels:** index of channel
- **mask:** mask image
- **histSize:** BIN count
- **ranges:** RANGE



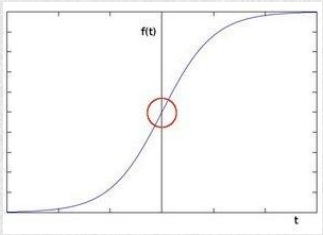
🧪 Exercise — Histogram of a Region

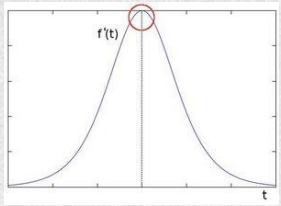
1. Create a binary mask that selects a rectangular region of the image.
The rectangular region (rows: 100 to 300 columns: 100 to 400)
2. Use the mask to:
    1. Visualize only that region.
    2. Compute its grayscale histogram.
3. Compare:
    1. The histogram of the full image
    2. The histogram of the masked region

Observe how the intensity distribution changes.

Remember: **cv2.calcHist()** allows you to pass a mask as input.
Use **cv2.bitwise_and()** to apply the mask

Edge Detection





The Sobel Operator detects edges marked by sudden changes in pixel intensity

X-Direction Kernel

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Y-Direction Kernel

$$\begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$G_x = A * I \qquad G_y = B * I$$

$$G = \sqrt{G_x^2 * G_y^2}$$

$$\Theta = arctan(G_y/G_x)$$

Filtering

2D convolution

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

cv2.filter2D()

- **src:** input image
- **depth:** -1 equal to input
- **kernel**

Filtering

2D convolution

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

cv2.filter2D()

- **src:** input image
- **depth:** -1 equal to input
- **kernel**

Averaging

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

cv2.blur()

- **src:** input image
- **kernel:** width and height

Filtering

2D convolution

$$K = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

cv2.filter2D()

- **src:** input image
- **depth:** -1 equal to input
- **kernel**

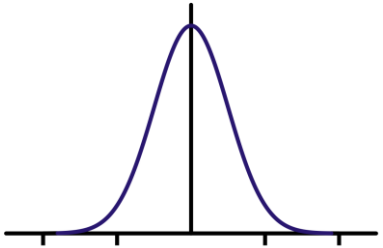
Averaging

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

cv2.blur()

- **src:** input image
- **kernel:** width and height

Gaussian Filter



cv.GaussianBlur()

- **src:** input image
- **kernel:** width and height
- **Standard deviation**

Morphological transformations

- Based on image shape
- Performed on binary images
- two inputs:
  - original image
  - kernel or structuring element

- **Erosion**: erodes away the boundary of the foreground object
- **Dilation**: opposite of erosion
- **Opening**: erosion followed by dilation
- **Closing**: dilation followed by erosion